

Rocket Digital Twin

Cyfrowy bliźniak rakiety wynoszącej satelitę na niską orbitę (LEO)

Dokumentacja techniczna

Zespół **SpatiumCavum** | Mateusz Bała

https://github.com/MateuszBala/Spaceshield_Hack_2026_PTR_DLD_Digital_Twin_Mission

Hackaton Space Shield 2026

Spis treści

1	Uruchomienie	2
1.1	Wymagania	2
1.2	Backend — API z silnikiem fizycznym	2
1.3	Frontend — interfejs przeglądarkowy	2
1.4	Weryfikacja poprawnego startu	2
2	Interfejs i obsługa	3
2.1	Widok główny: parametry, werdykt, sekwencja, telemetria	3
2.2	Wizualizacja toru lotu w przestrzeni	4
2.3	Przekrój boczny — opuszczanie atmosfery	4
2.4	Wykresy telemetrii w czasie	5
2.5	Sposób użycia: pętla eksperymentu	5
3	Podstawy fizyczne	5
3.1	Stan i układ odniesienia	6
3.2	Siły i równania ruchu	6
3.3	Werdykt o orbicie z elementów keplerowskich	7
3.4	Manewr wstawienia (gravity turn + Hohmann)	7
3.5	Napęd — model spójny z definicji	7
3.6	Co modelujemy, a co upraszczamy	8
4	Architektura i realizacja	8
4.1	Warstwy systemu	8
4.2	Trzy fazy jako reżimy numeryczne	8
4.3	Kluczowe decyzje projektowe	8
4.4	Endpointy API	9
4.5	Źródła	9

1. Uruchomienie

System składa się z dwóch procesów uruchamianych równolegle: **backendu** (API + silnik fizyczny, Python) oraz **frontendu** (interfejs przeglądarkowy, Node). Poniżej procedura krok po kroku.

1.1 Wymagania

- **Python** 3.13 oraz **uv** (menedżer środowisk i pakietów Python).
- **Node.js** 18+ oraz **npm** (dla frontendu).
- System operacyjny: Linux / macOS / WSL (testowano na Debianie).

1.2 Backend — API z silnikiem fizycznym

Z katalogu głównego repozytorium (workspace **uv** obejmuje pakiety **contracts**, **physics-engine**, **api**):

```
# 1. instalacja zaleznosci workspace (jednorazowo)
uv sync

# 2. uruchomienie serwera API na porcie 8000
uv run uvicorn dt_api.app:app --port 8000
```

Po uruchomieniu serwer nasłuchuje pod `http://localhost:8000`. Dokumentacja interaktywna API (Swagger) jest dostępna pod `http://localhost:8000/docs` — można tam wywołać endpoint `/simulate` bezpośrednio.

1.3 Frontend — interfejs przeglądarkowy

W osobnym terminalu, z katalogu **frontend/**:

```
# 1. instalacja zaleznosci npm (jednorazowo)
npm install

# 2. uruchomienie serwera deweloperskiego
npm run dev
```

Interfejs jest dostępny pod `http://localhost:5173`. Frontend komunikuje się z API przez proxy (`/api/*` → `localhost:8000`).

1.4 Weryfikacja poprawnego startu

Po otwarciu `http://localhost:5173` w prawym górnym rogu interfejsu widoczny jest wskaźnik źródła danych:

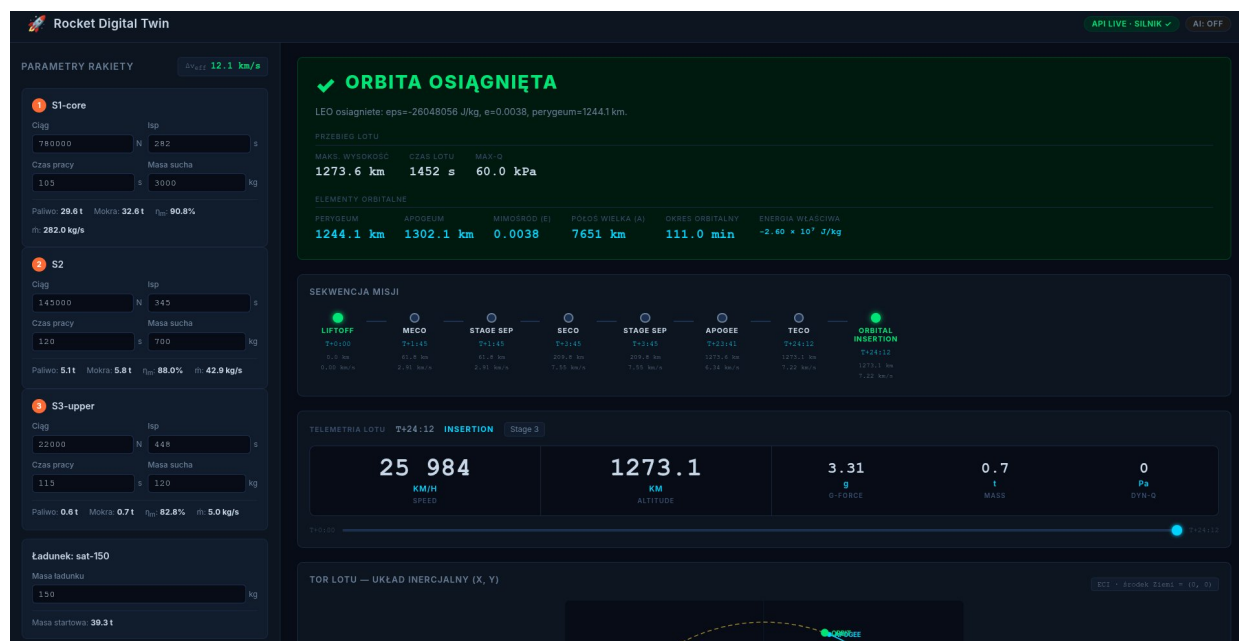
- **API LIVE · SILNIK ✓** — frontend połączony z API, wyniki pochodzą z prawdziwego silnika fizycznego (stan docelowy).
- **API LIVE · SILNIK stub** — API odpowiada, ale silnik niedostępny (zwracany jest wynik zastępczy).
- **API offline** — brak połączenia z API; interfejs działa na danych syntetycznych (tryb demonstracyjny offline).

Stan **API LIVE · SILNIK ✓** potwierdza, że cały łańcuch (interfejs → API → silnik → werdykt) działa poprawnie.

2. Interfejs i obsługa

Interfejs realizuje **pętlę Digital Twina**: użytkownik ustawia parametry rakiety, uruchamia symulację przyciskiem *Przelicz*, a system pokazuje werdykt o orbicie wraz z pełną wizualizacją lotu. Każdą zmianę można powtarzać dowolnie — to jest istota cyfrowego bliźniaka: eksperymentowanie na modelu przed realnym lotem.

2.1 Widok główny: parametry, werdykt, sekwencja, telemetria



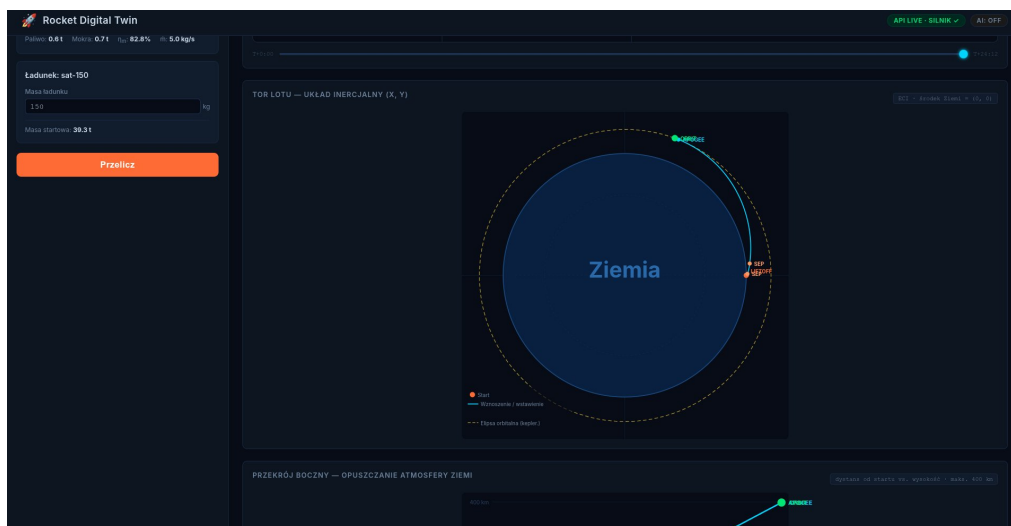
Rysunek 1: Widok główny — panel parametrów (po lewej), werdykt z elementami orbitalnymi, sekwencja misji oraz panel telemetrii.

Panel parametrów (lewa kolumna). Każdy stopień rakiety (S1, S2, S3) to karta z polami wejściowymi: *ciężar* [N], *impuls właściwy* [Isp] [s], *czas pracy* [s] i *masa sucha* [kg]. Pod polami interfejs pokazuje wielkości wyliczone: masę paliwa, masę mokrą, frakcję masową (η_m) i strumień masy. Niżej znajduje się ładunek (masa satelity) oraz łączna masa startowa. Wskaźnik Δv_{eff} u góry panelu to szacunkowy budżet prędkości. Na dole panelu — przycisk **Przelicz**, uruchamiający symulację.

Karta werdyktu. Po przeliczeniu pokazuje jednoznaczny wynik: **ORBITA OSIĄGNIĘTA** lub **BRAK ORBITY**, wraz z powodem w języku naturalnym (np. „LEO osiągnięte: ϵ , e , perygeum”). Sekcja *przebieg lotu* podaje maksymalną wysokość, czas lotu i max-Q; sekcja *elementy orbitalne* — perygeum, apogeum, mimośród, półosć wielką, okres i energię właściwą. Werdykt wynika z elementów keplerowskich (patrz sekcja 3), nie z chwilowej wysokości.

Sekwencja misji. Oś czasu kamieni milowych w stylu transmisji kosmicznych: LIFTOFF, MECO (wypalenie głównego stopnia), STAGE SEP (separacja), SECO (wypalenie drugiego stopnia), APOGEE (apogeum), ORBITAL INSERTION (wejście na orbitę). Przy każdym zdarzeniu podany jest czas, wysokość i prędkość.

Panel telemetrii (HUD). Duży, transmisyjny odczyt: prędkość [km/h], wysokość [km], przeciążenie [g], masa [t] i ciśnienie dynamiczne [Pa]. Suwak czasu (scrubber) pod panelem pozwala przewinąć lot do dowolnej chwili — odczyty i znacznik fazy (np. *INSERTION*, *Stage 3*) aktualizują się wraz z położeniem suwaka.

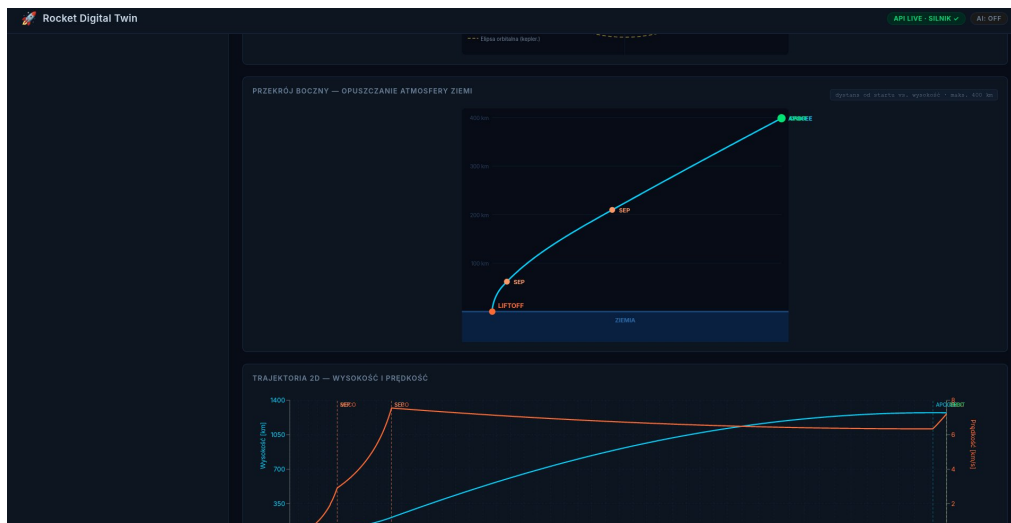


Rysunek 2: Tor lotu w układzie inercyjnym (X, Y) — Ziemia w środku, tor wznoszenia (linia ciągła) i pełna orbita keplerowska (linia przerywana).

2.2 Wizualizacja toru lotu w przestrzeni

Wykres **Tor lotu — układ inercyjny (X, Y)** pokazuje rzeczywisty kształt lotu wokół Ziemi (środek układu w punkcie $(0, 0)$). Linia ciągła to tor wznoszenia i wstawienia odtworzony z telemetry; linia przerywana to **pełna orbita** wyznaczona analitycznie z elementów keplerowskich — pokazuje, na jakiej orbicie satelita krąży po wejściu, mimo że symulacja kończy się w momencie wstawienia. Zaznaczone są zdarzenia (start, separacje, apogeum). Ten widok pokazuje, że system zna pełną geometrię orbity, nie tylko przebyty fragment toru.

2.3 Przekrój boczny — opuszczanie atmosfery



Rysunek 3: Przekrój boczny — wysokość w funkcji dystansu od startu (do 400 km), z fazą atmosferyczną i zdarzeniami separacji.

Wykres **Przekrój boczny** przedstawia wczesną fazę lotu: wysokość rakiety względem powierzchni Ziemi w trakcie opuszczania atmosfery (zakres do 400 km). Widoczna jest krzywizna toru (gravity turn) oraz zdarzenia separacji stopni — to ta część lotu, w której opór atmosfery jest istotny.

2.4 Wykresy telemetrii w czasie



Rysunek 4: Trajektoria 2D (wysokość i prędkość w czasie) oraz profil przeciążenia (g-force) z charakterystycznym „zębatym” kształtem.

Trajektoria 2D — wysokość i prędkość. Dwie krzywe w funkcji czasu: wysokość (oś lewa) i prędkość (oś prawa). Widoczne są punkty wypalenia stopni oraz apogeum. W fazie balistycznej (coast) wysokość rośnie ku apogeum, a prędkość maleje do minimum w apogeum — zgodnie z zasadą zachowania energii.

Profil przeciążenia (g-force). Przyspieszenie [g] w czasie. Charakterystyczny „zębaty” kształt: w trakcie pracy stopnia przyspieszenie rośnie (masa maleje przy stałym ciągu), spada do zera przy wypaleniu/separacji, po czym skacze przy zapłonie kolejnego stopnia. Ostatni skok odpowiada spalaniu górnego stopnia (circularyzacja na apogeum). Piki g-force pokrywają się ze zdarzeniami separacji — co potwierdza spójność telemetrii z sekwencją misji.

Ciśnienie dynamiczne (Q). Wielkość $Q = \frac{1}{2}\rho v^2$ w czasie — krytyczna dla obciążeń konstrukcji. Wyraźne maksimum (max-Q) we wczesnej fazie lotu to moment największego naprężenia aerodynamicznego; potem Q szybko spada wraz z rzednącą atmosferą.

2.5 Sposób użycia: pętla eksperymentu

1. Ustaw lub zmień parametr rakiety w panelu (np. czas pracy stopnia, ciąg, masę ładunku).
2. Kliknij **Przelicz**. System wywołuje silnik fizyczny przez API.
3. Odczytaj werdykt (orbita osiągnięta lub nie) oraz nową trajektorię, sekwencję i telemetrię.
4. Powtórz z inną wartością, aby zbadać wpływ decyzji projektowej na powodzenie misji.

3. Podstawy fizyczne

Silnik symulacji opiera się na mechanice orbitalnej problemu dwóch ciał. Źródła: wykłady MIT 16.346 *Astrodynamics* (elementy keplerowskie) oraz *Space Mission Analysis and Design* (SMAD) — stałe fizyczne i progi orbitalne. Zadanie dopuszcza uproszczone modele; każde uproszczenie jest tu świadomą decyzją inżynierską.



Rysunek 5: Komplet wykresów telemetrii: trajektoria, g-force oraz ciśnienie dynamiczne (Q) z wyraźnym maksimum ($max-Q$) we wczesnej fazie lotu.

3.1 Stan i układ odniesienia

Cały lot — od startu po orbitę — liczony jest w jednym inercyjnym, kartezjańskim układzie ze środkiem Ziemi w początku. Stan rakiety to pięć liczb:

$$\mathbf{s} = [x, y, v_x, v_y, m]$$

gdzie (x, y) to pozycja, (v_x, v_y) prędkość, m masa chwilowa. Lot rozpatrujemy w płaszczyźnie orbity (2D) — orbita jest płaska, a wszystkie istotne wielkości wynikają z tego stanu. Separacja stopnia jest nieciągłością wyłącznie w masie m ; pozycja i prędkość pozostają ciągłe.

3.2 Siły i równania ruchu

W fazach z napędem i atmosferą na raketę działają trzy siły:

Grawitacja — skierowana ku środkowi Ziemi:

$$\mathbf{g} = -\frac{\mu}{r^3} (x, y), \quad r = \sqrt{x^2 + y^2},$$

gdzie $\mu = 3,986 \times 10^{14} \text{ m}^3/\text{s}^2$ to stała grawitacyjna Ziemi.

Ciąg — wzdłuż osi rakiety, sterowany profilem kąta lotu. Masa maleje zgodnie ze zużyciem paliwa:

$$\frac{dm}{dt} = -\frac{F_{\text{ciąg}}}{I_{sp} g_0},$$

gdzie $g_0 = 9,80665 \text{ m/s}^2$ (standardowe przyspieszenie ziemskie).

Opór aerodynamiczny — przeciwny do prędkości:

$$D = \frac{1}{2} \rho(h) v^2 C_d A, \quad \rho(h) = \rho_0 e^{-h/H},$$

gdzie $\rho(h)$ to gęstość atmosfery malejąca wykładniczo z wysokością h , C_d współczynnik oporu, A powierzchnia odniesienia.

3.3 Werdykt o orbicie z elementów keplerowskich

To sedno modelu. **Osiągnięcie wysokości orbitalnej nie oznacza orbity** — można być na 400 km z prędkością skierowaną w górę i spaść z powrotem. O orbicie decydują elementy toru liczone z wektora stanu w momencie wstawienia.

Energia właściwa (całka energii):

$$\varepsilon = \frac{v^2}{2} - \frac{\mu}{r}.$$

Warunek orbity związanej: $\varepsilon < 0$ (rakietą nie ucieka).

Półos wielka z energii:

$$a = -\frac{\mu}{2\varepsilon}.$$

Wektor mimośrod (wektor Laplace’a; $h = xv_y - yv_x$ to moment pędu w 2D):

$$e_x = \frac{v_y h}{\mu} - \frac{x}{r}, \quad e_y = -\frac{v_x h}{\mu} - \frac{y}{r}, \quad e = \sqrt{e_x^2 + e_y^2}.$$

Mimośród e mówi, jak wydłużona jest orbita ($e = 0$ — okrąg).

Wysokość perygeum (najniższy punkt orbity nad powierzchnią):

$$h_p = a(1 - e) - R_{\oplus}, \quad R_{\oplus} = 6378 \text{ km}.$$

Kryterium sukcesu (progi ze SMAD, rozdz. 7.3):

$$\varepsilon < 0 \quad \wedge \quad h_p > 200 \text{ km} \quad \wedge \quad e < 0,25.$$

Czyli: orbita związana, perygeum powyżej gęstej atmosfery (stabilna LEO), mimośród ograniczony (orbita nie skrajnie eliptyczna).

3.4 Manewr wstawienia (gravity turn + Hohmann)

Wynoszenie na orbitę przebiega dwuetapowo. Najpierw rakietą wykonuje **gravity turn** — stopniowe pochylanie toru, by zamienić wznoszenie pionowe na prędkość poziomą. Następnie, po wypaleniu dolnych stopni, następuje **lot balistyczny (coast)** ku apogeum, a górny stopień odpala dopiero w pobliżu apogeum, wykonując **circularyzację** (manewr Hohmanna) — dodaje prędkość poziomą, podnosząc perygeum i domykając orbitę. Stąd na profilu g-force ostatni skok odpowiada spalaniu górnego stopnia, a prędkość osiąga minimum w apogeum tuż przed tym spalaniem.

3.5 Napęd — model spójny z definicji

Z parametrów stopnia (ciąg, I_{sp} , czas pracy) *wynikają* pozostałe wielkości, co uniemożliwia zadanie konfiguracji fizycznie sprzecznej:

$$\dot{m} = \frac{F_{\text{ciąg}}}{I_{sp} g_0}, \quad m_{\text{paliwo}} = \dot{m} \cdot t_{\text{pracy}}.$$

Budżet prędkości pojedynczej konfiguracji szacujemy równaniem Ciołkowskiego per stopień:

$$\Delta v = I_{sp} g_0 \ln \frac{m_{\text{początkowa}}}{m_{\text{końcowa}}}.$$

Impuls właściwy ograniczamy do $I_{sp} \leq 455$ s (twardy limit napędu chemicznego). Budżet do LEO to ok. 9,4 km/s ($\approx 7,7$ km/s prędkości orbitalnej + $\approx 1,75$ km/s strat).

3.6 Co modelujemy, a co upraszczamy

Modelujemy: ruch w polu grawitacyjnym (dwa ciała), ciąg i zużycie paliwa per stopień, opór atmosferyczny z wykładniczym modelem gęstości, separacje, max-Q, werdykt z pełnych elementów keplerowskich, 1–4 stopnie.

Upraszcza my świadomie: lot 2D zamiast 3D (orbita jest płaska); profil kąta lotu zamiast pełnej optymalizacji sterowania; wykładniczy model atmosfery zamiast warstwowego; pominięcie rotacji Ziemi, J_2 i oporu zależnego od kształtu. Uproszczenia dotyczą wierności modelu lotu, nie poprawności mechaniki orbitalnej, na której opiera się werdykt.

4. Architektura i realizacja

4.1 Warstwy systemu

System to monorepo podzielone na rozłączne pakiety wokół wspólnego **kontraktu danych**:

- **Silnik fizyczny** (`dt_physics`, NumPy/SciPy) — czysta numeryka, funkcja `simulate(SimRequest)` → `SimResult`. Nie zna sieci ani interfejsu; trywialny do testowania i ponownego użycia.
- **API** (`dt_api`, FastAPI) — cienka warstwa HTTP. Przyjmuje żądanie, woła silnik, zwraca wynik. Bez logiki fizycznej.
- **Frontend** (React + Vite + TypeScript) — interfejs; komunikuje się z backendem wyłącznie przez HTTP.
- **Kontrakt danych** (`dt_contracts`, Pydantic) — jedno źródło prawdy o kształcie danych (parametry, wynik, telemetria, elementy orbity). Typy interfejsu generują się z OpenAPI, eliminując rozjazd front↔backend.

Każda warstwa ma jedną odpowiedzialność i wyraźną granicę, co pozwoliło budować je równolegle. Granica HTTP istnieje tam, gdzie jest naturalna (przeglądarka ↔ serwer); wewnątrz backendu API woła silnik bezpośrednim wywołaniem funkcji.

4.2 Trzy fazy jako reżimy numeryczne

Faza	Dominująca fizyka	Zdarzenie kończące
Wznoszenie	ciąg + grawitacja + opór $\rho(h)$	max-Q, separacje, próg oporu
Wstawienie	ciąg + grawitacja (opór znikomy)	wypalenie / prędkość orbitalna
Orbita	dwa ciała, bez ciągu	werdykt keplerowski

Tabela 1: Trzy fazy lotu i ich charakterystyka numeryczna.

Przejścia między fazami to **wykrywane zdarzenia** (zera funkcji zdarzeń w solverze), nie sztywne progi czasu — dzięki czemu ich położenie nie zależy od gęstości kroku całkowania. Np. max-Q wykrywany jest jako moment zerowania pochodnej dQ/dt .

4.3 Kluczowe decyzje projektowe

- **Własny symulator zamiast OpenRocket** — OpenRocket nie zna pojęcia orbity ani wstawienia na LEO, a to jest sednem zadania. Własny silnik dał pełną kontrolę nad mechaniką orbitalną i werdyktem.
- **Werdykt z elementów keplerowskich, nie z wysokości** — najważniejsza decyzja fizyczna, odróżniająca projekt od kalkulatora wysokości.

- **Graceful degradation** — funkcje opcjonalne (np. optymalizacja AI) nie blokują rdzenia; API zgłasza dostępność, interfejs ukrywa niedostępne opcje.
- **Fallback Δv** — gdyby pełne całkowanie zawiodło, zdefiniowana jest jawna ścieżka awaryjna (budżet Ciołkowskiego), nie improwizacja.

4.4 Endpointy API

- POST /simulate — symulacja misji: `SimRequest` $\rightarrow$ `SimResult`.
- GET /capabilities — dostępność silnika i modułu AI, wersja kontraktu.
- GET /health — status serwera.
- POST /optimize — optymalizacja (zwraca 503, gdy moduł AI niedostępny).

4.5 Źródła

- MIT OpenCourseWare 16.346 *Astrodynamics* — problem dwóch ciał, wektor mimośrodów, całka energii, prawa Keplera.
- *Space Mission Analysis and Design* (SMAD), 3rd ed. — stałe Ziemi, g_0 , zakresy I_{sp} i frakcji masowych, progi orbitalne.